Лабораторная работа №13

Тема работы: «Разработка приложений создания и обработки динамических массивов»

1 Цель работы

Сформировать умения конструировать приложения с использованием различных способов описания динамических массивов, использовать различные способы резервирования и освобождения памяти, реализовывать сложные циклические алгоритмы обработки массивов.

2 Задание

1. Ввести матрицу порядка NxM. В каждой строке матрицы посчитать количество элементов кратных числу C и вывести номер строки с наибольшим количеством таких элементов.

12. Дана квадратная матрица порядка N. Найти максимальный элемент среди элементов, лежащих ниже побочной диагонали, и минимальный элемент среди элементов, лежащих выше главной диагонали.

14. Определить номера строк прямоугольной матрицы, хотя бы один элемент которых равен A, и элементы этих строк умножить на C.

3 Оснащение

ПК, задание по варианту, среда разработки Visual Studio.

4 Порядок выполнения работы

Код программы:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <climits>
using namespace std;

void zapoln(int rows, int cols, int** arr) {
    srand(time(0));
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            arr[i][j] = (rand() % 41) - 20;
        }
    }
}

void vivod(int rows, int cols, int** arr) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << arr[i][j] << '\t';
```

```
        }
        cout << endl;
    }
}

void func1(int rows, int cols, int** arr) {
    int* arr1 = new int[rows];
    int max = 0;
    int c;

    cout << "Введите число C: ";
    cin >> c;

    for (int i = 0; i < rows; i++) {
        int counter = 0;
        for (int j = 0; j < cols; j++) {
            if (arr[i][j] % c == 0) counter++;
        }
        cout << "В строке " << i + 1 << " количество элементов кратных " << c << ": " << counter << endl;
        arr1[i] = counter;
        if (counter > max) max = counter;
    }

    cout << "\nМаксимальное количество элементов кратных " << c << " = " << max << endl;
    cout << "Это количество встречается в строках: ";
    for (int i = 0; i < rows; i++) {
        if (arr1[i] == max) cout << (i + 1) << " ";
    }
    cout << endl;

    delete[] arr1;
}

void func2(int N, int** arr) {
    int maxBelowSecondary = INT_MIN;
    int minAboveMain = INT_MAX;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i + j > N - 1) {
                if (arr[i][j] > maxBelowSecondary) maxBelowSecondary = arr[i][j];
            }
            if (i < j) {
                if (arr[i][j] < minAboveMain) minAboveMain = arr[i][j];
            }
        }
    }

    cout << "\nМаксимальный элемент ниже побочной диагонали: " << maxBelowSecondary << endl;
    cout << "Минимальный элемент выше главной диагонали: " << minAboveMain << endl;
}

void func3(int rows, int cols, int** arr) {
    int A, C;
    cout << "Введите число A: ";
    cin >> A;
    cout << "Введите число C: ";
    cin >> C;

    cout << "Строки, содержащие элемент равный " << A << ":" << endl;
    for (int i = 0; i < rows; i++) {
        bool found = false;
        for (int j = 0; j < cols; j++) {
            if (arr[i][j] == A) {
                found = true;
                break;
            }
        }
        if (found) {
            cout << "Строка " << i + 1 << endl;
            for (int j = 0; j < cols; j++) {
                arr[i][j] *= C;
            }
        }
    }
```

```
 95. }
 96. int main() {
 97.     setlocale(LC_ALL, "RU");
 98.     srand(time(0));
 99.
100.     while (true) {
101.         cout << "\nВыберите задание:\n";
102.         cout << "1 - Количество элементов кратных C по строкам\n";
103.         cout << "2 - Макс. ниже побочной и мин. выше главной диагонали\n";
104.         cout << "3 - Умножение строк, содержащих A, на C\n";
105.         cout << "0 - Выход\n";
106.
107.         int choice;
108.         cin >> choice;
109.         if (choice == 0) break;
110.
111.         if (choice == 1) {
112.             int rows, cols;
113.             cout << "Введите число строк: ";
114.             cin >> rows;
115.             cout << "Введите число столбцов: ";
116.             cin >> cols;
117.
118.             if (rows <= 0 || cols <= 0) {
119.                 cout << "Размерность неверная!" << endl;
120.                 continue;
121.             }
122.
123.             int** arr = new int* [rows];
124.             for (int i = 0; i < rows; i++) arr[i] = new int[cols];
125.
126.             zapoln(rows, cols, arr);
127.             vivod(rows, cols, arr);
128.             func1(rows, cols, arr);
129.
130.             for (int i = 0; i < rows; i++) delete[] arr[i];
131.             delete[] arr;
132.         }
133.         else if (choice == 2) {
134.             int N;
135.             cout << "Введите порядок квадратной матрицы N: ";
136.             cin >> N;
137.
138.             if (N <= 0) {
139.                 cout << "Размерность неверная!" << endl;
140.                 continue;
141.             }
142.
143.             int** arr = new int* [N];
144.             for (int i = 0; i < N; i++) arr[i] = new int[N];
145.
146.             zapoln(N, N, arr);
147.             vivod(N, N, arr);
148.             func2(N, arr);
149.
150.             for (int i = 0; i < N; i++) delete[] arr[i];
151.             delete[] arr;
152.         }
153.         else if (choice == 3) {
154.             int rows, cols;
155.             cout << "Введите число строк: ";
156.             cin >> rows;
157.             cout << "Введите число столбцов: ";
158.             cin >> cols;
159.
160.             if (rows <= 0 || cols <= 0) {
161.                 cout << "Размерность неверная!" << endl;
162.                 continue;
163.             }
164.
165.             int** arr = new int* [rows];
166.             for (int i = 0; i < rows; i++) arr[i] = new int[cols];
167.
168.             zapoln(rows, cols, arr);
169.             cout << "Исходная матрица:" << endl;
170.             vivod(rows, cols, arr);
```

```
171.
172.            func3(rows, cols, arr);
173.
174.            cout << "Матрица после преобразования:" << endl;
175.            vivod(rows, cols, arr);
176.
177.            for (int i = 0; i < rows; i++) delete[] arr[i];
178.            delete[] arr;
179.        }
180.        else {
181.            cout << "Неверный выбор задания!" << endl;
182.        }
183.    }
184.
185.    return 0;
186. }
187.
```

Результат программы:

```
Выберите задание:
1 - Количество элементов кратных C по строкам
2 - Макс. ниже побочной и мин. выше главной диагонали
3 - Умножение строк, содержащих A, на C
0 - Выход
1
Введите число строк: 5
Введите число столбцов: 4
-17     5       18      13
-3      12      -17     8
-2      5       9       11
-13     -13     19      -2
1       17      -9      -2
Введите число C: 5
В строке 1 количество элементов кратных 5: 1
В строке 2 количество элементов кратных 5: 0
В строке 3 количество элементов кратных 5: 1
В строке 4 количество элементов кратных 5: 0
В строке 5 количество элементов кратных 5: 0

Максимальное количество элементов кратных 5 = 1
Это количество встречается в строках: 1 3
```

Рисунок 1 - Пример выполнения задачи №1

```
Выберите задание:
1 - Количество элементов кратных C по строкам
2 - Макс. ниже побочной и мин. выше главной диагонали
3 - Умножение строк, содержащих A, на C
0 - Выход
2
Введите порядок квадратной матрицы N: 5
3       12      18      -8      -11
11      13      -11     -7      -19
-8      20      -20     4       -1
12      5       -4      -5      1
-14     1       4       -12     -9

Максимальный элемент ниже побочной диагонали: 4
Минимальный элемент выше главной диагонали: -19
```

Рисунок 2 - Пример выполнения задачи №12

```
Выберите задание:
1 - Количество элементов кратных C по строкам
2 - Макс. ниже побочной и мин. выше главной диагонали
3 - Умножение строк, содержащих A, на C
0 - Выход
3
Введите число строк: 5
Введите число столбцов: 10
Исходная матрица:
3       3       -12     1       7       0       -10     -14     -16     -3
0       -14     20      10      19      19      -10     0       -13     1
-15     6       -11     12      15      7       7       -1      -9      -14
-15     -3      -11     -18     -20     -7      -3      -14     1       18
20      13      -13     -15     -17     13      -8      -4      12      12
Введите число A: 3
Введите число C:
4
Строки, содержащие элемент равный 3:
Строка 1
Матрица после преобразования:
12      12      -48     4       28      0       -40     -56     -64     -12
0       -14     20      10      19      19      -10     0       -13     1
-15     6       -11     12      15      7       7       -1      -9      -14
-15     -3      -11     -18     -20     -7      -3      -14     1       18
20      13      -13     -15     -17     13      -8      -4      12      12
```

Рисунок 3 – Пример выполнения задачи №14

5 Контрольные вопросы:

1. **Дайте краткое понятие динамического массива в контексте программирования на языке C++.**

Динамический массив — это массив, размер которого задаётся во время выполнения программы, а память для него выделяется в куче с помощью операторов new или функций стандартной библиотеки.

2. **Раскройте значение выделения памяти под динамический массив и освобождения этой памяти.**

Выделение памяти позволяет создать массив нужного размера во время работы программы, а освобождение (delete[]) предотвращает утечки памяти и обеспечивает эффективное использование ресурсов.

3. **Сформулируйте сущность работы с указателями при обработке динамических массивов.**

Указатели хранят адрес начала массива и позволяют обращаться к его элементам через арифметику указателей или оператор индексации.

4. **Изложите особенности работы с динамическими массивами по сравнению со статическими.**

– Размер задаётся во время выполнения, а не компиляции.

– Хранятся в куче, а не в стеке.

– Требуют явного освобождения памяти.

– Более гибкие, но менее безопасные без контроля.

5. **Раскройте особенности обработки ошибок при работе с динамическими массивами в C++.**

Основные ошибки — утечки памяти, обращение к освобождённой памяти, выход за границы массива. Для их предотвращения используют проверки индексов, RAII-классы (например, std::vector) и умные указатели.